

Purpose

This script computes TPM (Transcripts Per Million) from raw gene counts. It expects STAR's ReadsPerGene.out.tab (unstranded counts) and a table of gene lengths in base pairs. Output is a two-column TSV: gene, TPM.

Pipeline Overview

- Load raw unstranded gene counts from STAR ReadsPerGene.out.tab (column 2).
- Load per-gene lengths (in base pairs).
- Compute length-normalized counts ($CPK = \text{count} / \text{length}[\text{kb}]$).
- $\text{TPM per gene} = (CPK / \text{sum}(CPK)) \times 1e6$.
- Write final TSV with columns: gene, TPM.

Inputs & Outputs

Inputs (via Snakemake)

- counts_file: STAR ReadsPerGene.out.tab (TSV; gene_id, unstranded, forward, reverse). Script uses column 2 (unstranded) and skips STAR summary rows (comment='_').
- lengths_file: TSV with columns: gene, length (length in base pairs).

Output

- out_file (TPM TSV): columns gene, TPM.

Computation Details

For each gene g with raw count C_g and length L_g (bp):

- 1) Length-normalized count (CPK): $CPK_g = C_g / (L_g / 1000)$
- 2) TPM per sample: $TPM_g = (CPK_g / \text{sum}_i CPK_i) \times 1,000,000$

Rationale: Longer genes accumulate more reads. Normalizing by gene length adjusts for this effect, then TPM rescales to a constant library size for comparability.

Script Interface (Snakemake)

```
# Snakemake-injected variables
counts_file = snakemake.input.counts      # STAR ReadsPerGene.out.tab
lengths_file = snakemake.input.lengths    # gene lengths TSV (gene, length)
out_file = snakemake.output.tpm           # output TSV (gene, TPM)
```

Core Steps (Pseudo-code)

```
counts = read_table(counts_file, sep='\t', comment='_', usecols=[0,1])
counts.columns = ['gene', 'count']
lengths = read_table(lengths_file, sep='\t') # columns: gene, length
df = merge(counts, lengths, on='gene')
df = df[df['length'] > 0]
df['norm'] = df['count'] / (df['length'] / 1000)
df['TPM'] = (df['norm'] / df['norm'].sum()) * 1e6
df[['gene', 'TPM']].to_csv(out_file, sep='\t', index=False)
```

Usage

README - compute_tpm.py

Snakemake Rule Example

```
rule compute_tpm:
    input:
        counts = 'star/{sample}.ReadsPerGene.out.tab',
        lengths = 'ref/gene_lengths.tsv'
    output:
        tpm = 'quant/{sample}.tpm.tsv'
    conda:
        'envs/rnaseq.yml'
    shell:
        'python compute_tpm.py'
```

Direct Run (for testing)

If testing outside Snakemake, adapt the script to read CLI arguments:

```
python compute_tpm.py \
    --counts star/S1.ReadsPerGene.out.tab \
    --lengths ref/gene_lengths.tsv \
    --out quant/S1.tpm.tsv
```

Expected File Formats

STAR ReadsPerGene.out.tab

- No header; 4 columns: gene_id, unstranded, forward, reverse.
- Rows starting with '_' or '__' are STAR summary lines and are ignored.

Gene Lengths TSV

- Tab-separated with columns: gene, length (bp).
- Gene IDs must match the count table gene IDs (same versioning).

Troubleshooting

- Empty output: ensure gene IDs overlap between counts and lengths.
- Zero-division: filter out genes with length = 0 (already handled).
- Unexpected scaling: confirm lengths are in base pairs.
- Version mismatch: strip/add Ensembl version consistently upstream.

Notes

- Unstranded counts (column 2) are used for general compatibility; swap to forward/reverse if needed.
- TPM is relative within a sample; not a substitute for differential testing.
- This step outputs normalized expression for visualization or correlation.